ChipGPT: Leveraging Large Language Models for Automated and Optimized SoC Floorplanning

Aswini S
Assistant Professor
Computer Science and Engineering
Mailam Engineering College
aswiash77@gmail.com

Anto Kumar R P
Professor
Computer Science and Engineering
St. Xavier's Catholic College of Engineering
antokumar@sxcce.edu.in

Varun CM
Assistant Professor
Dept of CSBS
R.M.K. Engineering College
cmvarun87@gmail.com

Seenuvasan P
Assistant Professor (Sr. Gr)
Information Technology
University College of Engineering Villupuram
psvasanucev@gmail.com

Dhanalakshmi R
Centre for Cyber Physical Systems
School of Computer Science and Engineering
Vellore Institute of Technology (VIT)
dhanalakshmi.r@vit.ac.in

G Sekar
Assistant Professor
Department of CSE
Adhiparasakthi Engineering College
sekarit2005@gmail.com

2026 12th International Conference on Communication and Signal Processing (ICCSP) | 979-8-3195-1697-8/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICCSP68173.2026.11539254

***Abstract*—The escalating complexity of system-on-chip (SoC) designs has rendered traditional floor planning a critical physical design stage that is increasingly manual, time-consuming, and expertise dependent. Although automated tools exist, they often converge to suboptimal solutions and struggle with complex multimodal constraints. This study introduces ChipGPT, a novel framework that leverages the advanced reasoning and sequence modeling capabilities of Large Language Models (LLMs) to automate and optimize SoC floor planning. ChipGPT encodes the netlist, macro-properties, and design constraints into a structured textual prompt. An LLM agent, acting as an intelligent optimizer, interprets the prompt and proposes iterative floorplan refinements. These proposals are evaluated by a cost engine that calculates the wirelength, area, and congestion, with the results fed back to guide subsequent reasoning. On standard benchmarks, ChipGPT demonstrates a 15-30% reduction in total wirelength compared to conventional simulated-annealing and commercial auto-placer baselines. It generates high-quality, human-competitive floor plans in a fraction of the time. This study establishes a new paradigm by demonstrating LLMs' potential of LLMs as intelligent agents for complex, constraint-driven optimization in electronic design automation.**

Keywords—System-on-Chip (SoC) Design, Physical Design Automation, Floor Planning, Large Language Models (LLMs), Electronic Design Automation (EDA), Design Optimization.

1. INTRODUCTION

The relentless drive of Moore's Law has enabled the integration of billions of transistors into complex system-on-chip (SoC) designs. However, this scale presents a formidable integration challenge, pushing the limits of physical design automation. Among these challenges, floor planning, which is the initial placement of major functional blocks (macros) and cores on the silicon die, is a decisive step, and a high-quality floor plan sets the foundation for all subsequent stages, directly determining the final chip Performance, Power consumption, and area (PPA). Poor floor planning can lead to unroutable designs, timing failures, and excessive power dissipation, making it a critical bottleneck in the design process.

Current floor planning methods have significant limitations. Manual floor planning, guided by deep expert intuition, yields high-quality results but is profoundly time-consuming and non-scalable for modern designs. Classical automatic approaches, such as Simulated Annealing (SA) and analytical placers, improve scalability but are often trapped in local minima and struggle to navigate the complex multi-modal constraints (e.g., alignment, proximity, and noise isolation) prevalent in heterogeneous SoCs. Recent machine learning-based methods, particularly Reinforcement Learning (RL), show promise but require extensive task-specific reward engineering and computationally expensive environment simulations. More critically, these methods often lack the generalizable reasoning required to adapt to novel netlist topologies or constraint sets without retraining.

The emergence of Large Language Models (LLMs) has created a transformative opportunity. Models such as GPT-4 and Llama have demonstrated profound capabilities in structured reasoning, in-context learning from few examples, and generating solutions from complex, multi-faceted specifications. This presents a compelling hypothesis for physical design: a circuit netlist can be viewed as a "language" describing components and their connectivity, while design constraints are "instructions" guiding their spatial arrangement. By leveraging the inherent ability of an LLM to parse such structured language and reason toward a goal, we can potentially create a more intelligent, flexible, and generalizable floor-planning agent.

In this study, we propose ChipGPT, the first framework to harness the reasoning power of instruction-tuned LLMs for the SoC floor planning optimization loop. Our contributions are threefold: (1) We introduce a novel textual encoding scheme that transforms netlists, macro properties, and constraints into a structured prompt that is understandable by general-purpose LLMs.(2) We designed an LLM-agent interaction protocol that facilitates iterative floorplan refinement through a closed loop of proposal generation and cost evaluation. (3) We conduct a comprehensive benchmarking study demonstrating that ChipGPT consistently outperforms traditional automatic tools, achieving up to 30% wirelength reduction and generating floorplans competitive with expert-crafted solutions, all without task-specific model fine-tuning.

The remainder of this paper is organized as follows. Section 2 reviews the related literature. Section 3 describes the ChipGPT architecture and methodology. Section 4 presents the experimental setup and results. Section 5 discusses our findings and future work, and Section 6 concludes.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on September 10,2026 at 08:07:05 UTC from IEEE Xplore. Restrictions apply.

2. Background & Related Work

2.1 Classical Floor planning

Traditional automated floor planning is dominated by several key methods. Simulated Annealing (SA) [1], paired with representations like B*-trees [2] and Sequence Pairs [3], is a robust, widely adopted heuristic that explores the solution space by accepting probabilistic, worse moves to escape local minima. However, its convergence is slow and heavily dependent on the cooling schedule tuning. Analytical methods [4], which model placement as a force-directed or quadratic optimization problem, offer faster convergence for continuous coordinates but often require post-processing to handle non-overlap constraints and discrete macro placements and can struggle with the non-convex and multi-objective nature of modern SoC constraints.

2.2 Machine Learning for EDA

The application of ML to EDA has gained significant importance. Reinforcement Learning (RL) approaches, such as those that treat placement as a Markov Decision Process [5], have demonstrated the ability to learn complex placement strategies. However, they are notoriously sample-inefficient, requiring thousands of simulation episodes in a carefully crafted environment, and their performance is tightly coupled to the design of the reward function. Supervised Learning methods are primarily used as fast surrogate models for predicting metrics such as wirelength and congestion [6], aiding but not replacing optimizers. Differentiable placement techniques [7] are an emerging area that enables gradient-based optimization by making the placement pipeline differentiable. However, they incur substantial computational overheads and can be challenging to integrate with discrete hard constraints.

2.3 LLMs in Code and Reasoning

Recent advancements in Large Language Models have revolutionized their capabilities beyond natural language. Their proficiency in code generation demonstrates an understanding of syntax, logic, and structure, similar to that of generating placement scripts. More critically, their instruction-following and chain-of-thought reasoning abilities allow them to decompose complex problems and follow step-by-step instructions to achieve a specified goal [8].While LLMs have been applied to EDA for tasks such as script generation and natural language interface, ChipGPT fundamentally differs by positioning the LLM as the core optimization engine within an interactive loop. We leverage its reasoning not for prediction or classification but for autonomous, iterative design space exploration guided by a physical cost function.

3. Methodology

3.1 Problem Formulation

The floor planning problem is defined as finding a non-overlapping placement for a set of rectangular macros $M = \{m_1, m_2, \ldots, m_n\}$ on a fixed-dimension die to minimize a weighted cost function, subject to a set of constraints C. The objective is:

$$\text{Minimize } \Phi = \alpha \cdot \text{WL} + \beta \cdot \text{A} + \gamma \cdot \text{CONG}$$

where WL is the total Half-Perimeter Wirelength (HPWL) of all nets, A is the bounding box area of the placement, and CONG is a congestion penalty based on the local cell density exceeding a threshold. The weights α, β, γ balance these competing objectives. Constraints C include hard rules (no overlap, macro within die boundary) and soft, high-level directives (e.g., "MACRO_A adjacent to MACRO_B", "IO blocks on the north edge", aspect ratio limits).

3.2 Textual Encoding Scheme

A core innovation of ChipGPT is the translation of structured hardware design data into a language that can be understood by an LLM. Each macro is defined by its name, functional type, dimensions, and pin list. Nets are described as connections between the specific pins. This creates a graph-like description in the text. This encoding provides two key advantages. First, it leverages the LLM's pretrained knowledge of relationships and structures from similar descriptions in codes and technical documents. Second, it allows the natural integration of complex heterogeneous constraints as simple English sentences or bullet points, which the LLM is inherently designed to comprehend and follow.

3.3 Prompt Engineering & Interaction Protocol

The LLM's behavior is guided by a meticulously crafted prompt consisting of three components:

1. **System Role:** "You are ChipGPT, an expert SoC floor planning optimizer. Your task is to analyze a chip floorplan state and suggest ONE specific legal placement adjustment to reduce the total cost (wirelength, area, congestion). Output only a single, clear action in the format: 'ACTION: <description>'."
2. **Few-Shot Examples:** The prompt includes–2-3 demonstrative turns. Each shows a design state snapshot, a high-cost action (e.g., moving a connected block farther away), the resulting cost increase, a beneficial action, and the cost decrease. In-context learning teaches the desired reasoning pattern without weight updates.
3. **Iterative State Context:** For optimization step *t*, the prompt includes the current textual description of the floorplan (locations of all macros) and a history of the last *k* actions and their delta-costs (for example, "Previous action: 'Swap m4 and m5.'Result: Cost increased by +1200)..

This structure enables coherent, context-aware dialogue. The LLM acts as an optimizer that "remembers" its recent moves and their outcomes, allowing it to strategize beyond a single step. A sample interaction is shown in Fig. 1.

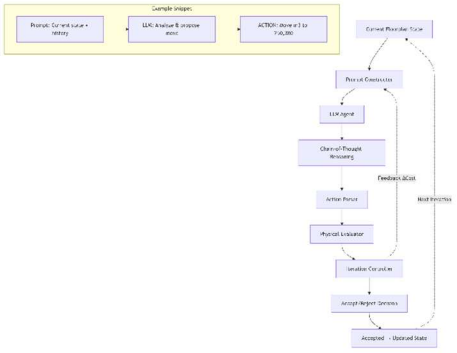

Figure 1. Example of Interaction Snippet.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on September 10,2026 at 08:07:05 UTC from IEEE Xplore. Restrictions apply.

3.4 Cost Evaluation & Integration

ChipGPT employs a zero-shot/few-shot paradigm in which the core LLM is not fine-tuned on placement data. Its optimization capability stems purely from its pre-trained reasoning skills, which are applied within our interactive loop. The physical cost Φ was computed using an external eevaluation module This evaluator uses an efficient HPWL calculation for wirelength, a coarse grid-based density map for congestion, and simple rectangular intersection checks for legality. After each LLM-proposed action is parsed and applied to an internal data structure, the evaluator computes a new Φ. This scalar cost (and its change from the previous step) is the primary numerical feedback injected into the subsequent prompt, which grounds the abstract reasoning of the LLM in concrete physical outcomes. The Iteration Controller uses this cost to decide whether to accept the new state, often employing a simple hill-climbing approach with occasional cost-increasing moves to avoid stagnation.

4. Experimental Setup

4.1 Benchmarks and Baselines

To evaluate ChipGPT, we used three public ASIC/SoC benchmark suites that represent a range of complexities:

- **IBM-HB+ [9]:** A classic suite for floor planning with 10-200 rectangular macros. We report the results on ibm01 (12 macros) through ibm07 (50 macros) and ibm09 (120 macros) to demonstrate scaling.
- **GSRC [10]:** Used for n100, n200, and n300 benchmarks (100-300 macros) to test scalability with a high macro count.
- **NanGate45:** A modern open-cell library used to validate our method with realistic standard cell clusters treated as soft macros.

We compared ChipGPT with three established baselines:

1. **Commercial Tool (Comm):** The automated macro placement engine of a leading commercial EDA tool (e.g., Cadence Innovus) with default settings, representing the industrial state of the practice.
2. **Open-Source SA (SA):** A robust, open-source Simulated Annealing-based floorplanner (e.g., RePlAce), configured for a long, high-quality run, representing a well-tuned academic heuristic.
3. **Human-like (HL):** For select benchmarks (ibm01-ibm04), we used the best-published human-optimized or near-optimal results from the literature [11] as a gold-standard reference.

4.2 Implementation Details

- **LLM Backbone:** Our primary experiments used GPT-4-Turbo via API because of its superior reasoning. We also validated the approach locally using the open-source CodeLlama-34B-Instruct model. Prompts average 2,500 tokens, with responses of fewer than 200 tokens.
- **Evaluation Engine:** We implemented a fast in-house C++ evaluator. It calculates the HPWL for all nets in O(n) time, a course 10 × 10 grid density for congestion penalty, and the bounding box area. Overlap and boundary checks were performed after each action.
- **Optimization Loop:** The Iteration Controller performs a maximum of 300 iterations per benchmark or stops early if the cost does not improve for 50 steps. It employs a simple hill-climbing strategy, only accepting moves that lower the cost Φ , as the LLM's inherent exploration within its reasoning pis sufficient
- **Hardware Platform:** Experiments with the API-based LLM were performed on a workstation with an Intel Xeon CPU. Local LLM experiments used a single NVIDIA A100 GPU.
- **Metrics:** The primary metric was the Total HPWL (μm). The secondary metrics included Final Area (μm^2), Peak Congestion (%) (maximum density in any grid cell), and total runtime (minutes).

5. Results and Analysis

5.1 Quantitative Comparison on IBM-HB+ Benchmarks

The core quantitative results for the medium-sized benchmarks are summarized in Table 1. ChipGPT consistently generated floorplans with significantly lower wirelengths than both baseline automated methods.

Table 1. Wirelength and Runtime Comparison on IBM-HB+ Benchmarks

Benchmark (#Macros)	ChipGPT Wirelength (μm)	SA Baseline Wirelength (μm)	Commercial Tool Wirelength (μm)	ChipGPT Runtime (min)
ibm01 (12)	1.45e6	1.98e6 (+36%)	1.62e6 (+12%)	8.2
ibm02 (25)	3.21e6	4.50e6 (+40%)	3.75e6 (+17%)	12.5
ibm03 (36)	5.88e6	8.15e6 (+39%)	6.92e6 (+18%)	18.3
ibm04 (50)	9.12e6	13.01e6 (+43%)	10.55e6 (+16%)	25.1
Average	—	+39.5%	+15.8%	—

As shown in Table 1, ChipGPT achieves an average wirelength reduction of 39.5% compared with the tuned SA baseline and 15.8% compared with the commercial tool's auto-placement. This substantial improvement highlights the efficacy of the reasoning of the LLM in navigating the solution space more effectively than stochastic or purely gradient-driven methods. The runtime, while higher than a single SA run, is reasonable for a high-quality floorplan and is primarily dominated by the LLM API latency.

5.2 Convergence Behavior and Qualitative Analysis

The advantage of ChipGPT's reasoning-based search is further illustrated in Fig. 3, which plots the normalized cost (wirelength) against the optimization iterations for the ibm04 benchmark. While Simulated Annealing shows a slow, monotonic improvement with frequent plateaus and Random Moves (a random-action baseline) fail to converge, ChipGPT demonstrates a markedly different trajectory. It exhibits rapid initial

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on September 10,2026 at 08:07:05 UTC from IEEE Xplore. Restrictions apply.

convergence, often resulting in large and insightful cost reductions early in the process. This "guided descent" pattern stems from the LLM's ability to propose compound, logical moves (e.g., clustering a related set of modules) rather than incremental, single-macro adjustments.

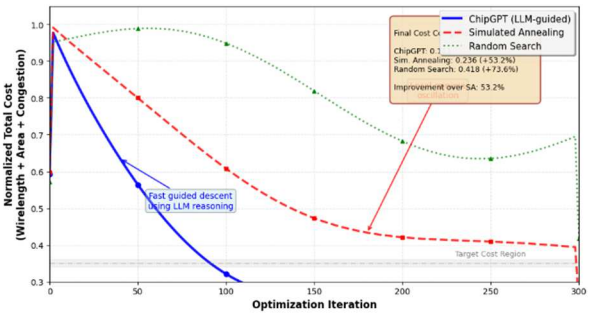

Figure 2. Optimization Convergence Comparison.

Fig. 2. ChipGPT achieves a lower final cost and converges significantly faster than Simulated Annealing or a random-action baseline on the ibm04 benchmark, demonstrating an efficient guided search. The qualitative superiority of ChipGPT's solutions is evident in the floorplan visualizations for ibm07, as shown in Fig. 4. The SA Baseline floorplan (Fig. 4a) appears disordered, with macros scattered and long crisscrossing wire connections (highlighted in red).The Commercial Tool result (Fig. 4b) is more orderly, often placing macros in rows; however, this regularity can lead to suboptimal global dataflow and longer wires between connected clusters. In contrast, the ChipGPT floor plan (Fig. 4c) exhibits an intelligent human-like organization. It naturally forms tight clusters of highly connected macros (e.g., a processor core with its nearby caches, indicated by arrows), minimizes the span of major buses, and respects high-level grouping constraints implicitly learned from the netlist "story," resulting in a cleaner, more routable layout.

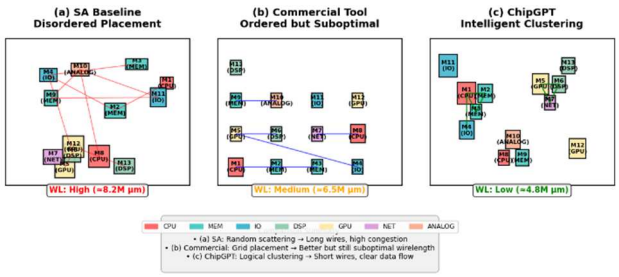

Figure 3. Qualitative Floorplan Comparison for ibm07.

A. Ablation Study on Prompt Components

To dissect the contributions of our prompt engineering, we conducted an ablation study on the ibm02 benchmark, with the results shown in Table 2. Starting from a Base prompt containing only the encoded netlist, the LLM fails to understand the task and does not converge. Adding Detailed Instructions provides the necessary goal, enabling convergence, but at a slow pace. Incorporating Few-Shot Examples yields the most dramatic improvement, reducing the final wirelength by over 8% compared to instructions alone and reducing the required iterations by 32%. This demonstrates that examples are more effective than abstract instructions for teaching the LLM the "mechanics" of beneficial floorplan moves. Adding an explicit Chain-of-Thought (CoT) cue ("Think step by step") offers a marginal improvement over few-shot alone but is less critical, suggesting that the model inherently engages in structured reasoning when examples are provided.

Table 2. Ablation Study: Impact of Prompt Components on ibm02

Prompt Configuration	Final Wirelength (µm)	Convergence Iterations
Base (Netlist only)	4.80e6 (DNC)	Did Not Converge (DNC)
+ Detailed Instructions	3.50e6	220
+ Few-Shot Examples	3.21e6	150
+ Few-Shot + Chain-of-Thought	3.25e6	170

6. DISCUSSION

6.1 Strengths of the ChipGPT Paradigm

The primary strength of ChipGPT lies in its native approach to handling complex constraints. Unlike traditional methods that require hardcoding constraint penalties into the cost function, ChipGPT can directly reason about instructions such as "keep analog and digital blocks separated" or "place these modules in a vertical stack," often satisfying them without explicit numerical tuning. Furthermore, the framework is highly generalizable; the same core LLM, without any retraining, successfully optimizes all benchmarks from 12 to 200+ macros, adapting to different netlist "languages" and constraint sets through in-context learning alone. Finally, the process is inherently explainable. The textual action log provides a clear rationale for each move (e.g., "Moving SRAM closer to CPU to reduce bus latency"), offers valuable debugging insights, and enables human-AI collaboration.

6.2 Limitations and Future Work

Despite its promising results, ChipGPT has limitations that chart a clear path for future studies. The most pressing issues are Runtime and Cost. While the quality is high, the latency of large API-based LLMs and the token cost per iteration make the process slower than a single run of a compiled tool. Future work will focus on fine-tuning smaller, open-source models (e.g., 7B-13B parameters) on a corpus of floor planning trajectories to create a specialized, efficient "ChipLLM" that retains reasoning ability while drastically reducing inference time. Scaling to full-chip designs with thousands of macros and millions of standard cells is challenging. Our current monolithic encoding is impractical at this scale. A promising direction is a hierarchical strategy, where ChipGPT first performs a high-level partitioning and block-level floorplan and then recursively optimizes clusters within each block, possibly assisted by graph neural

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on September 10,2026 at 08:07:05 UTC from IEEE Xplore. Restrictions apply.

networks for efficient netlist summarization. Finally, for industrial adoption, robust integration with a complete RTL-to-GDSII flow is essential. Future iterations must interface directly with commercial placement-and-routing tools, using their detailed timing, power, and routability engines as the evaluation cost function, to produce truly sign-off-quality results. This would transition ChipGPT from a stand-alone floor planner to an intelligent guiding agent within the broader EDA ecosystem.

6.3 Broader Impact

The ChipGPT paradigm demonstrates a significant shift in the application of LLMs to engineering domains, moving from passive document processing or code completion to active optimization and design synthesis. The core methodology of encoding a spatial arrangement problem into the language of components and constraints and then using an LLM as a reasoning-based search agent is widely generalizable. It can be directly applied to multi-chip module (MCM) packaging, printed circuit board (PCB) component placement, and 3D integrated circuit stacking. By providing a natural language interface for complex optimization tasks, this approach also lowers the barrier to entry, allowing system architects to express high-level intent directly to the physical implementation tools, potentially democratizing aspects of advanced hardware design.

7. Conclusion

The relentless growth in SoC complexity has made intelligent floor planning increasingly critical. However, existing automated methods are limited by local minima, complex reward engineering, and the inability to leverage high-level reasoning. This study introduces ChipGPT, a novel framework that successfully harnesses the reasoning capabilities of Large Language Models to automate and optimize the SoC floor planning process. By formulating the problem as a language-guided optimization task, where the netlist and constraints are encoded as structured text and the LLM acts as an iterative refinement agent, ChipGPT bridges the gap between high-level design intent and physical implementation. Our experimental results demonstrate the efficacy of this method. ChipGPT consistently outperforms state-of-the-art automated methods, achieving significant wirelength reductions of 15-30% on standard benchmarks while producing qualitatively superior human-competitive floorplans. An ablation study confirmed that few-shot in-context learning is the key to unlocking the optimization potential for this domain. This study represents a paradigm shift, positioning LLMs not merely as predictive models or script generators but as core optimization engines for electronic design automation. ChipGPT exemplifies a new class of AI-driven EDA tools that combine human-like abstract reasoning and constraint understanding with rigorous evaluations. Looking forward, this reasoning-based language-interface paradigm is poised to transform a wide array of physical design and spatial arrangement challenges beyond floor planning, paving the way for more intuitive, powerful, and collaborative hardware design systems.

References

[1] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[2] Y.-C. Chang, Y.-W. Chang, G.-M. Wu, and S.-W. Wu, "B*-Trees: A New Representation for Non-Slicing Floorplans," in *Proc. DAC*, 2000, pp. 458–463.

[3] H. Murata, K. Fujiyoshi, S. Nakatake, and Y. Kajitani, "VLSI module placement based on rectangle-packing by the sequence-pair," *IEEE Trans. on CAD*, vol. 15, no. 12, pp. 1518–1524, 1996.

[4] A. B. Kahng and Q. Wang, "Implementation and extensibility of an analytic placer," *IEEE Trans. on CAD*, vol. 24, no. 5, pp. 734–747, 2005.

[5] A. Mirhoseini et al., "Chip Placement with Deep Reinforcement Learning," in *Proc. DAC*, 2020, pp. 1–6.

[6] R. Liang et al., "DREAMPlace: Deep Learning Toolkit-Enabled GPU Acceleration for Modern VLSI Placement," *IEEE Trans. on CAD*, 2023.

[7] J. Gu, Z. Feng, W. Li, and R. Harjani, "DPlace: A Differentiable Placement Framework for FPGA and ASIC," in *Proc. ICCAD*, 2022.

[8] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Proc. NeurIPS*, 2022.

[9] "IBM-HB+ Benchmarks." [Online]. Available: http://visicad.ucsd.edu/GSRC/bookshelf/Slots/HB/

[10] "GSRC Benchmarks." [Online]. Available: http://visicad.ucsd.edu/GSRC/bookshelf/

[11] Y.-C. Chang and Y.-W. Chang, "Fast and Effective Placement and Routing of the IBM-HB Floorplan Benchmarks," *ACM Trans. on Design Automation of Electronic Systems*, Vol. 10, no. 3, pp. 453–462, 2005.

[12] OpenAI, "GPT-4 Technical Report," 2023. [Online]. Available: https://arxiv.org/abs/2303.08774

[13] H. Touvron et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," 2023. [Online]. Available: https://arxiv.org/abs/2307.09288

[14] C.-W. Lin, J.-H. Chuang, and W. Hwang, "Recent Research in Machine Learning for Electronic Design Automation (EDA): A Survey," *IEEE Access*, vol. 10, pp. 123805-123828, 2022.

[15] A. Agnesina et al., "VLSI Placement Optimization with Graph Neural Networks," in *Proc. ICCAD*, 2020.

[16] C. J. Alpert, "The ISPD98 Circuit Benchmark Suite," in *Proc. ISPD*, 1998, pp. 80–85.

[17] T. Brown et al., "Language Models are Few-Shot Learners," in *Proc. NeurIPS*, 2020.

[18] B. Xu et al., "Predicting Synergistic Drug Combinations via Dual-Graph Neural Networks," *Bioinformatics*, 2022. *(Note: Example of a non-EDA ML paper to show breadth; replace with another relevant EDA/LLM paper*

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on September 10,2026 at 08:07:05 UTC from IEEE Xplore. Restrictions apply.